universität innsbruck

Pull-Out Test Dejori – Comparison MOOSE and Abaqus

Dipl.-Ing. Manuel Hradsky, BSc

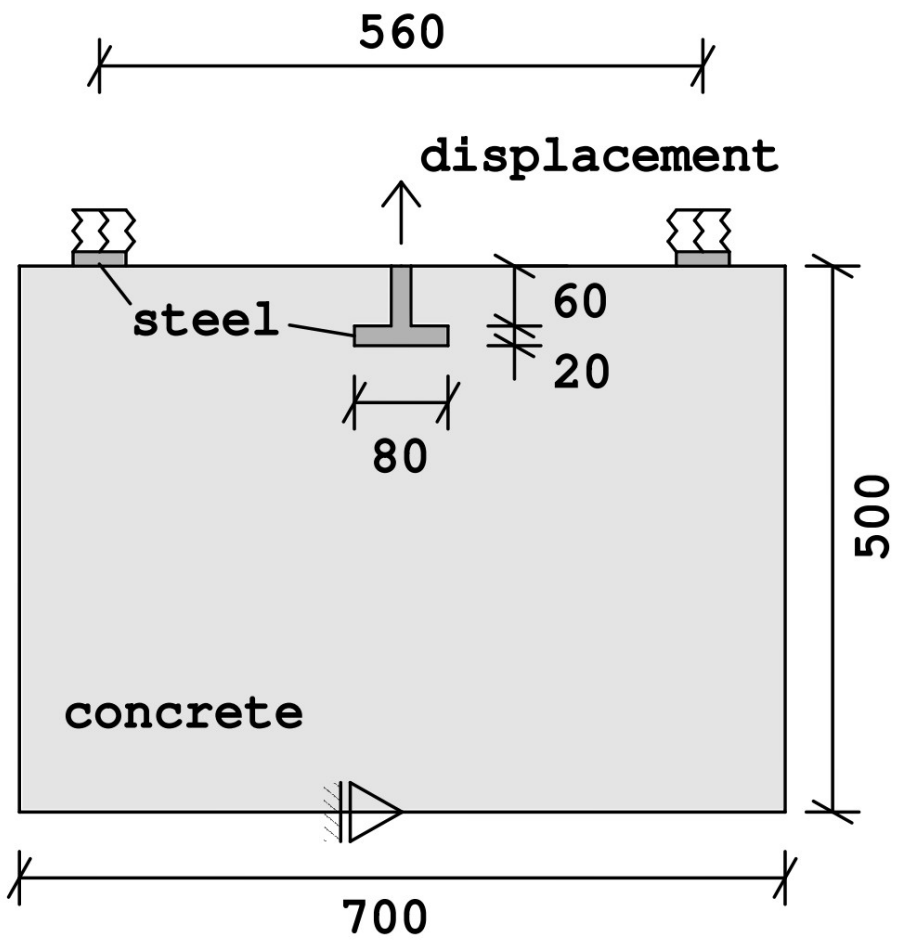
Unit of Strength of Materials and Structural Analysis

Dejori Test Set-Up

- Test Specimen: 700 mm (x-axis) × 500 mm (y-axis)
- Element Thickness: 4 mm (z-axis)

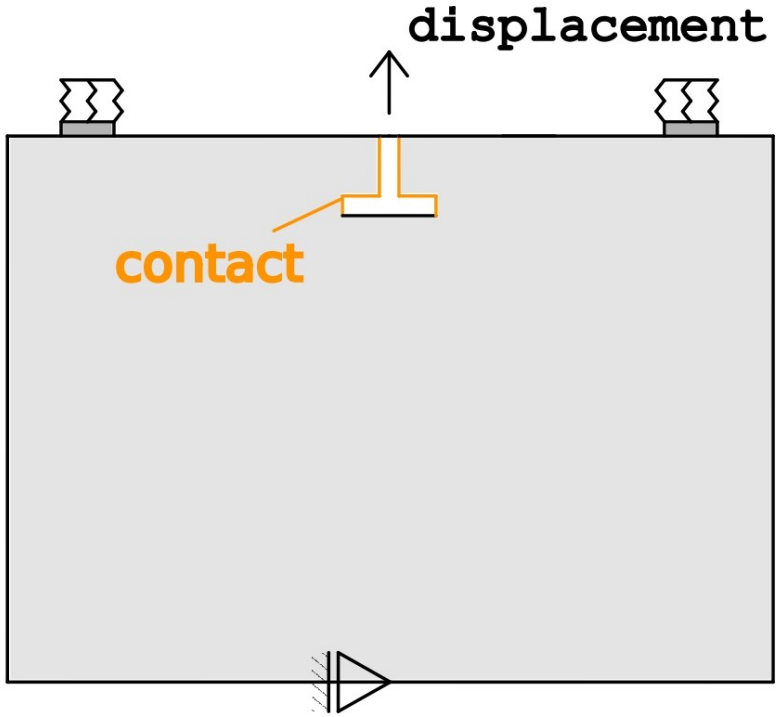
- Horizontal Restraint: Single node at the center of the bottom edge constrained in x-direction
- Out-of-Plane Constraint: Displacement in z-direction fixed for the entire body
- Lateral Supports: Elastic foundation applied to the top left and right steel support surfaces

- Prescribed Displacement: Vertical load applied as a controlled displacement at the top surface of the steel anchor
- Data Extraction: Resulting displacement is calculated as an average value at the bottom edge of the steel anchor head



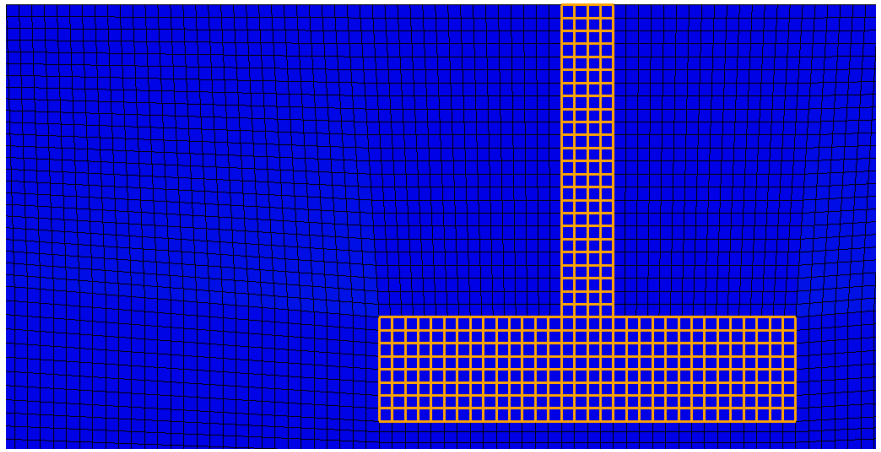
Dejori Test Set-Up

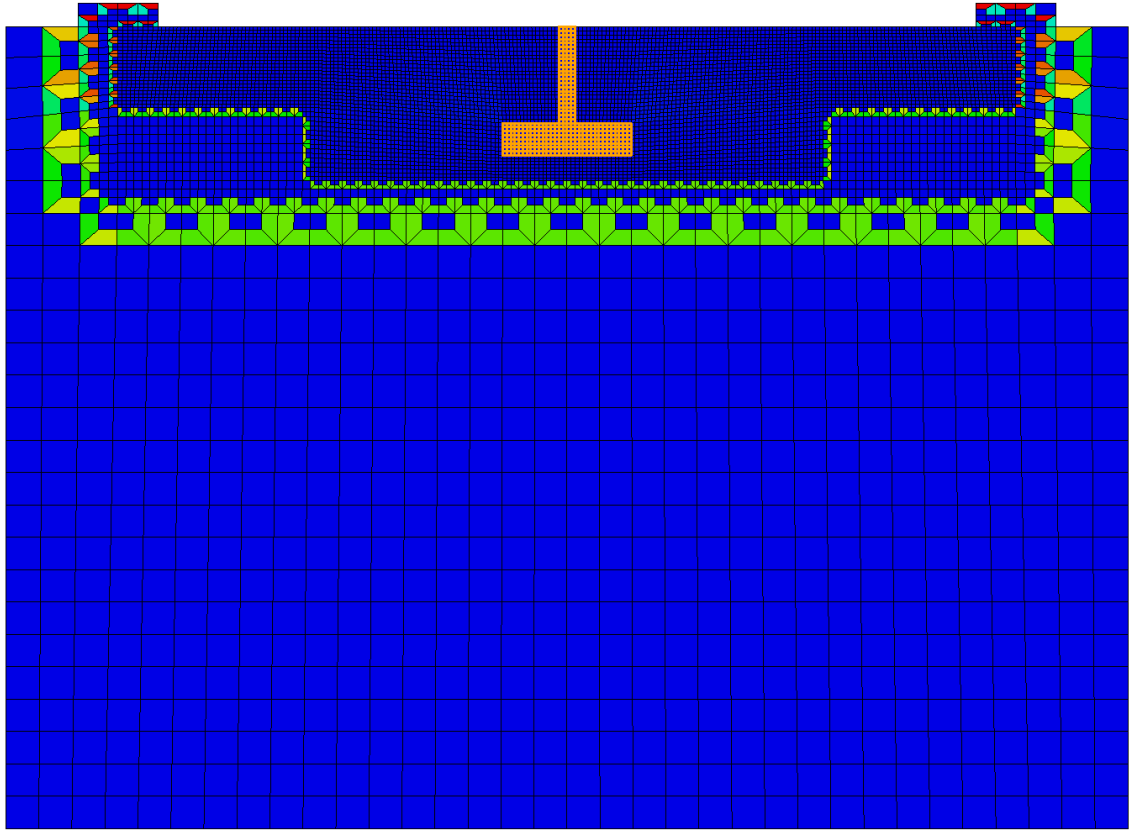
- **Anchor Shaft:** Contact defined on all vertical surfaces
- **Anchor Head:** Contact applied to both vertical and top surfaces
- **Interface:** Full surface-to-surface contact modeled between components



Simulation Mesh

- Preprocessing: Mesh generation performed using Cubit
- Local Refinement: Increased mesh density applied around the steel anchor and towards the supports
- Element Sizing: Targeted element edge length of 2.5 mm within the refined zones





MOOSE Input File - Physics & Solid Mechanics

```
[Physics]
  [SolidMechanics]
    [Dynamic]
      [volumetric_domain]
        block = '1 2 3'
        strain = SMALL
        add_variables = true
        use_automatic_differentiation = true
        incremental = true
        generate_output = 'stress_xx stress_yy stress_zz vonmises_stress strain_xx strain_yy strain_zz'
        save_in = 'force_x force_y force_z'
        hht_alpha = -0.414214
      []
    []
  []
[]
```

- **Physics Block:** Serves as the container for defining the governing physical laws and equations
- **SolidMechanics Module:** Is a library of simulation tools that solve continuum mechanics problems
- **Dynamic Formulation:** Accounts for inertial effects, mass and acceleration

MOOSE Input File - Physics & Solid Mechanics

```
[Physics]
  [SolidMechanics]
    [Dynamic]
      [volumetric_domain]
        block = '1 2 3'
        strain = SMALL
        add_variables = true
        use_automatic_differentiation = true
        incremental = true
        generate_output = 'stress_xx stress_yy stress_zz vonmises_stress strain_xx strain_yy strain_zz'
        save_in = 'force_x force_y force_z'
        hht_alpha = -0.414214
      []
    []
  []
[]
```

- **Automatic Differentiation (AD):** Leverages AD to provide exact Jacobians, significantly enhancing the robustness and convergence of the Newton solver
- **Incremental Updates:** Essential for path-dependent behavior and numerical stability during complex contact interactions (not necessary right now, will become necessary with more complex materials)

MOOSE Input File - Materials

- Right now, only linear elastic materials are used for a first comparison

- **Materials Block:** Defines the constitutive laws and physical properties assigned to specific mesh subdomains
- **Elasticity Block:** Computes an isotropic elasticity tensor through the Young's modulus and the Poisson's ratio
- **Density Block:** Needed for the dynamic analysis, the material property density gets a value assigned
- **Stress Block:** Computes the elastic stress for a total and small strain formulation

```
[Materials]
  [elasticity_steel]
    type = ADComputeIsotropicElasticityTensor
    block = 'steel_anchor steel_supports'
    youngs_modulus = 200000.0
    poissons_ratio = 0.3
  []
  [density_steel]
    type = ADGenericConstantMaterial
    block = 'steel_anchor steel_supports'
    prop_names = 'density'
    prop_values = 7.85e-9
  []
  [stress_steel]
    type = ADComputeLinearElasticStress
    block = 'steel_anchor steel_supports'
  []
```

MOOSE Input File – Boundary Conditions

- **Predescribed Displacement** (pull_steel):
  - Vertical displacement
  - Loading via a predefined function (linear)
  - Configuration preset=false: satisfied during the non-linear Newton-Raphson iterations rather than being forced before the step begins
- **Elastic Support** (foundation_y):
  - Elastic constraint (surface based, same as foundation in Abaqus)
  - Penalty method: Acts like a spring
  - Bedding stiffness: 100N/mm^3 iteratively fitted in Abaqus and copied here to match the linear-elastic slope of the experimental tests

```
[BCs]
  [pull_steel]
    type = ADFunctionDirichletBC
    variable = disp_y
    boundary = 'load'
    function = smooth_step
    preset = false
  []

  [foundation_y]
    type = ADPenaltyDirichletBC
    variable = disp_y
    boundary = 'bc_left_surface bc_right_surface'
    value = 0.0
    penalty = 100
  []
[]
```

MOOSE Input File – Contact

```
[Contact]
  [contact_left_stud_vertical]
    primary = 'cont_surf_stl_l_stud_vert'
    secondary = 'cont_surf_conc_l_stud_vert'
    formulation = mortar_penalty
    penalty = 1e5
    model = coulomb
    friction_coefficient = 0.35
  []
[]
```

- **Contact Block:** Can be used to specify parameters related to mechanical contact enforcement in MOOSE simulations
- **Surface Assignment:** Primary surface (stiffer material) and secondary surface
- **Formulation:** mortar_penalty
  - Mortar contact: segment to segment, typically works with Lagrange Multipliers
  - Penalty: used instead of LM for better convergence
- **Friction Model:** Coulomb Law with a coefficient of 0.35

MOOSE Input File – Executioner

```
[Executioner]
  type = Transient

  [Quadrature]
    order = SECOND
  []

  end_time = 60.0

  automatic_scaling = true

  [TimeIntegrator]
    type = NewmarkBeta
    beta = 0.500000
    gamma = 0.914214
  []

  [TimeStepper]
    type = IterationAdaptiveDT
    dt = 1e-2
    cutback_factor = 0.5
    growth_factor = 1.5
    optimal_iterations = 10
  []
[]
```

- **Executioner Block:** Describes how the simulation will be executed

- **Type transient:** Executioner for time varying simulations
- **Quadrature:** Increases or decreases the precision of volume and surface integrals within the elements, for hex20 elements second order equals reduced integration (2x2x2)
- **Conditioning:** Automatic scaling rescales the Jacobian matrix internally to stabilize the solver while dealing with high stiffness contrasts of the entries
- **Time Integration:** Newmark-Beta, an implicit, second-order integration method for structural dynamics (values taken from Abaqus)
- **Adaptive Stepping:** the time step reacts to the solvers performance

MOOSE Input File - Physics & Solid Mechanics

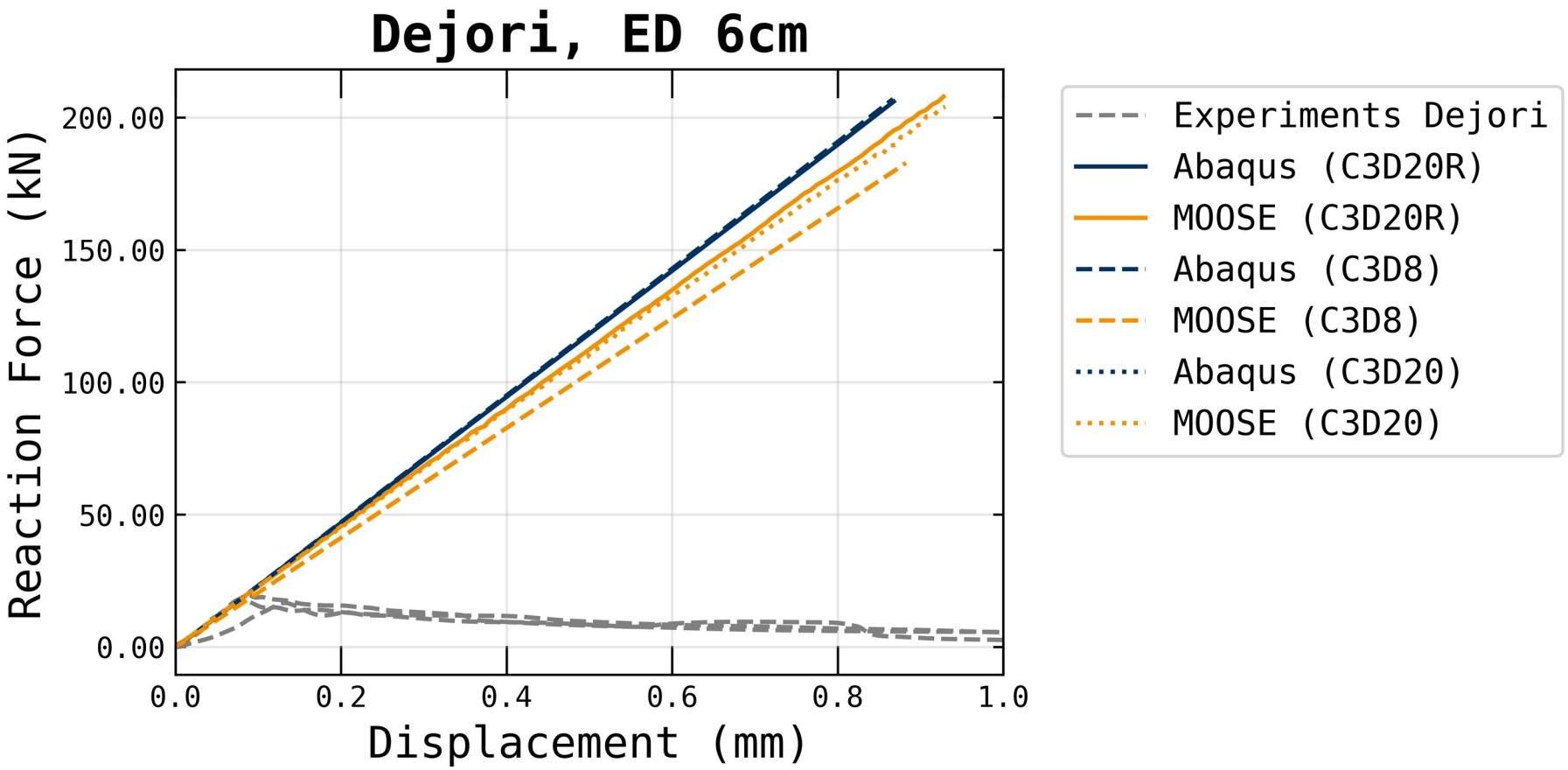
```
[Executioner]
  dtmax = 0.5

  solve_type = 'NEWTON'

  petsc_options_iname = '-pc_type -pc_factor_mat_solver_type -snes_linesearch_type'
  petsc_options_value = 'lu       mumps                      basic'

  nl_abs_tol = 1e-3
  nl_rel_tol = 5e-3

  l_max_its  = 100
  nl_max_its = 50
[]
```

- **Solve Strategy:** Uses a Newton-Raphson iteration scheme
- **Linear Solver (PETSc Options):** Uses a direct solver while utilizing MUMPS (Multifrontal Massively Parallel sparse direct Solver) library
- **Line Search:** Performs a standard Newton step without additional damping or backtracking (can be changed if the simulation becomes unstable)

Post Processing - Displacement field

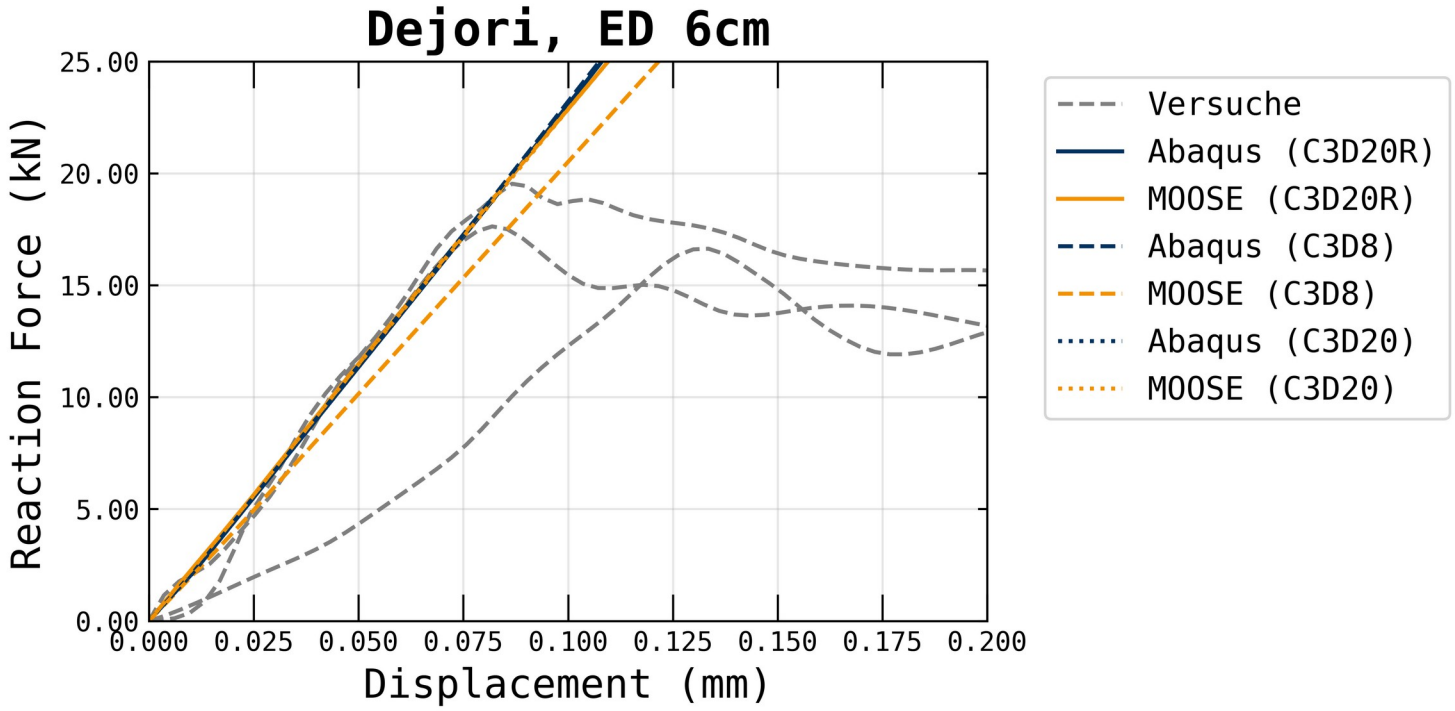
- Output Formats: Exodus file for 3D visualization and CSV file for defined postprocessors
- Direct analysis of vertical displacement (y-direction) is possible via Paraview

Post Processing – Load-Displacement Curve

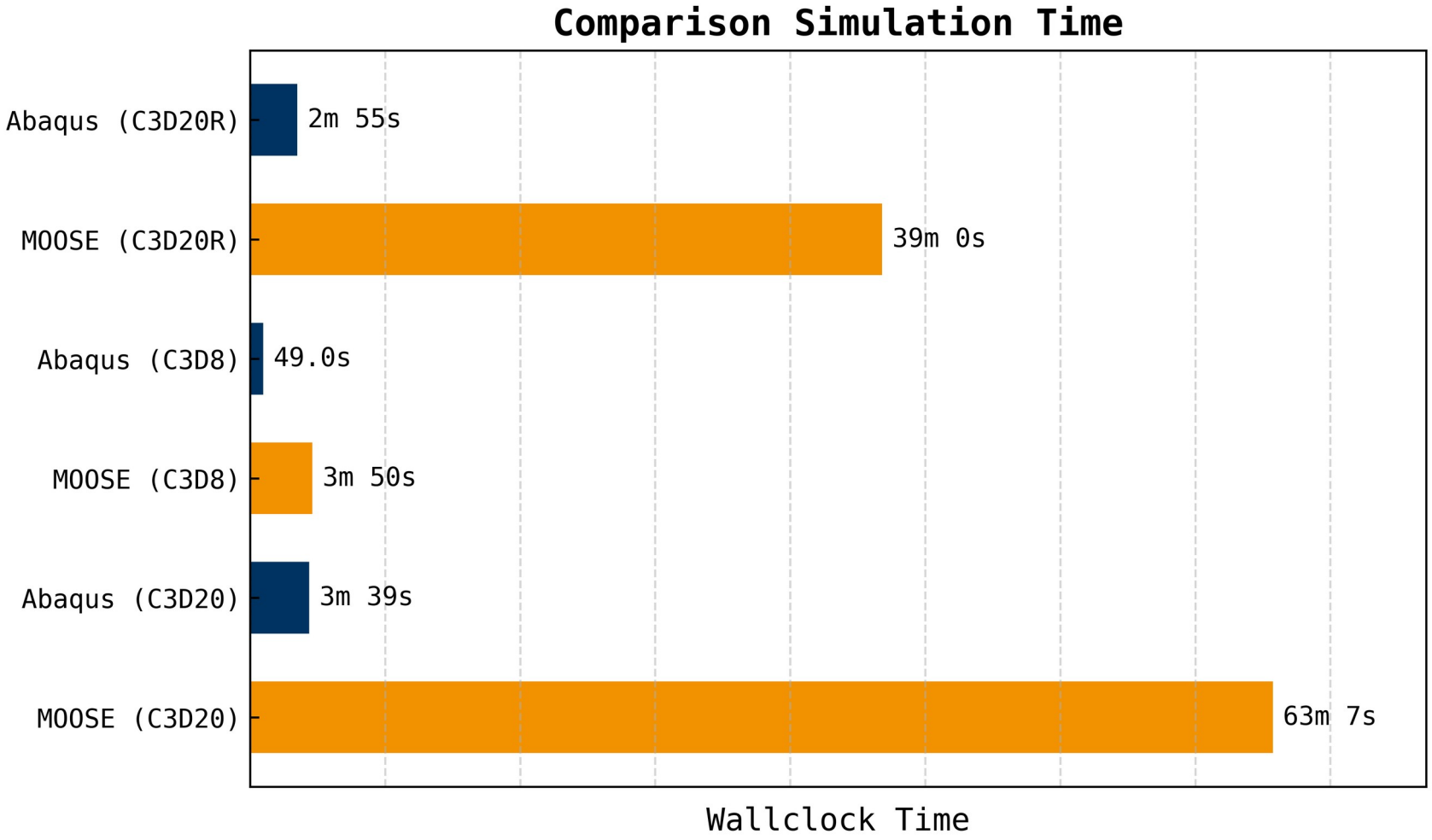


Post Processing – Load-Displacement Curve

- **Abaqus:** Consistent load-displacement curves across all three tested element types
- **MOOSE:** Noticeable discrepancy between linear (Hex8) and quadratic (Hex20) element formulations
- Comparable results for Abaqus and MOOSE



Post Processing – Simulation Time



Post Processing – Simulation Time

```
SIMULATION PERFORMANCE COMPARISON
=====================================================================
Simulation Name | Increments   | Iterations   | Wallclock Time (s)
---------------------------------------------------------------------
Abaqus C3D20R   | 129          | 137          | 175
MOOSE C3D20R    | 128          | 127          | 2340.20
Abaqus C3D8     | 129          | 141          | 49
MOOSE C3D8      | 128          | 127          | 230.48
Abaqus C3D20    | 129          | 137          | 219
MOOSE C3D20     | 128          | 127          | 3787.34
```

- Almost the same amount of increments and iterations between Abaqus and MOOSE
- **HOWEVER:** Significantly longer simulation times in MOOSE

Post Processing – Simulation Time

```
Performance Graph:
-----------------------------------------------------------------------------------------------------------------------------------
|                                Section                                | Calls |   Self(s)  |   Avg(s)   |  Total(s)  |   Avg(s)   |    %
-----------------------------------------------------------------------------------------------------------------------------------
|     Transient::PicardSolve                                            |   128 |      2.557 |      0.020 |   2240.263 |     17.502 |  99.07
|       FEProblem::solve                                                |   128 |    272.806 |      2.131 |   2168.764 |     16.943 |  95.91
|         Console::outputStep                                           |   255 |      0.016 |      0.000 |      0.016 |      0.000 |   0.00
|         NonlinearSystemBase::initialBCs                               |   128 |      1.245 |      0.010 |      1.245 |      0.010 |   0.06
|         FEProblem::computeJacobianInternal                            |     1 |      0.000 |      0.000 |      2.731 |      2.731 |   0.12
|           FEProblem::computeUserObjects                               |     1 |      0.350 |      0.350 |      0.350 |      0.350 |   0.02
|         FEProblem::computeJacobianInternal                            |   127 |      0.003 |      0.000 |   1357.349 |     10.688 |  60.03
|           FEProblem::computeUserObjects                               |   127 |    105.802 |      0.833 |    105.802 |      0.833 |   4.68
|         FEProblem::computeResidualInternal                            |   255 |      0.006 |      0.000 |    534.502 |      2.096 |  23.64
|           FEProblem::computeUserObjects                               |   255 |     64.475 |      0.253 |     64.475 |      0.253 |   2.85
|       FEProblem::computeUserObjects                                   |   128 |      2.402 |      0.019 |      2.402 |      0.019 |   0.11
```

- **Primary Time Sinks:** The element-level calculations completely dominate the simulation time, accounting for over 84% of the total effort
- **Jacobian Assembly (~61%):** Formulating the global stiffness matrix requires ~1357 seconds
- **Residual Calculation (~24%):** Evaluating internal element and contact forces at the integration points takes ~535 seconds.

Thank you for your attention!